

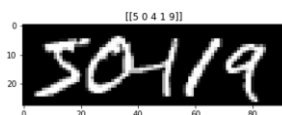
Practical work 3:

Embedded training of continuous Neuro-HMM

training without symbol segmentation

In this session we focus on moving from a pure HMM to a neuro_HMM. We focus on images of strings of digits that are of size $28 \times L$ as shown on figure below. $L < 140$ is the image width in pixels and may vary from one image to another because the digits may overlap, which makes the segmentation almost impossible without recognition.

The ground truth is the sequence of classes and does not provide any information about the position of the symbols in the image. Thus, training will proceed without segmentation with embedded training. Contrary to previous experiments, we will now consider the pixel columns of the image as the input frames of the sequences, we give up with discrete HMM.



Example of a 5 digits sequence generated from the MNIST dataset.

Neuro HMM architecture

A neuro HMM is a HMM where the emission probabilities are modeled with a neural network. A Multi-Layer Perceptron (MLP) is chosen, with a softmax activation function at its output, which estimates the state posterior probabilities:

$$\gamma_t(s_k) = P(q_t = s_k | O)$$

Every character Neuro_HMM will share the same Neural Network, which all in all makes a number of classes for the MLP equal to:

$$N_classes = n_symbols \times n_states$$

with $n_symbols = 10$ and $n_states = 5, \dots, 10, \dots, 15$

The Neuro-HMM equation is the following, where we see the state posteriors divided by the state priors to provide a normalized likelihood of the observation (in red). The transition probabilities and Initial and final probabilities (in blue) remain similar to those of a HMM

$$P(O, Q | M) \propto \pi_{q_1} \prod_t \frac{P(q_t | o_t)}{P(q_t)} \times P(q_t | q_{t-1})$$

This equation can simplify to the following one if we consider equi-probable state priors:

$$P(O, Q|M) \propto \pi_{q_1} \prod_t P(q_t|o_t) \times P(q_t|q_{t-1})$$

Embedded training with a Neuro HMM

We proceed using the following algorithm where one can see the use of **mini-batch** for training the neuro-hmm.

Also notice that this training scheme **only trains the Neural Network**, whereas the transition probabilities remain unchanged during training. The Markov Model remains unchanged with a uniform distribution for the left-right model (see the initialization procedure of the Neuro-HMM)

```
Models : the liste of digit models (the ten digit models)
NN = the initialized MLP
for iteration = 1,...,Max_EM_iter do
    for each mini_batch do
        EXPECTATION STEP
        for n = 1,...,mini_batch size do
            GT_Model = Create HMM for GT of example n°n
            Compute gamma probabilities for this example with GT_Model
            Derive the gamma probabilities for the 10 digit models
            Concatenate every frames of the batch
            Concatenate every frame predictions of the batch
        MAXIMISATION STEP
            Train the MLP to predict the Neuro-HMM predictions

    Compute train_LogLikelihood
    Compute valid_LogLikelihood

    if valid_LogLikelihood > previous_valid_LogLikelihood then
        Store the NN parameters
```

The **Embedded_Training()** method in **neuro_hmm.py** implements this algorithm.

Question 1 : explain the following lines of code

```
# shuffle the training data at each iteration

# Pairs each training sequence with its corresponding ground truth label
l=list(zip(train_data,gt_train))

# Randomly reorders these pairs
random.shuffle(l)

# Unzips the shuffled pairs back into separate lists
train_data, gt_train = list(zip(*l))
```

This is crucial because it will prevent the model from learning ordered patterns and it ensures each batch contains diverse examples.

Question 2 : in the Embedded_Training() method, what does the M step?

The M step computes a better model by improving the loss using standard backpropagation on the training data and the predictions from the E-step. Therefore, E-step uses current MLP to generate the most likely predictions and compute the LL, while the M step updates the MLP to better match those expectations, while HMM structure stays constant throughout training.

Question 3 : explain the pred_on_batch() method

The pred_on_batch() method is used on every iteration of E-step during training to estimate emission probabilities (make predictions) using the current model and it applies Forward-Backward algorithm to calculate the log likelihood.

The method begins by iterating through each sequence in the input batch, where for every sequence it constructs a composite HMM that represents the ground truth digit sequence. This composite model, referred to as GT_Model, is dynamically assembled by concatenating the individual HMMs corresponding to each digit in the ground truth sequence. The neural network component, represented by my_mlp, generates emission probabilities for the current sequence.

```
GT_Model = GT_HMM(config['n_states'],gt,Models)
```

During the initial training iteration, these probabilities are initialized uniformly to provide an equiprobable starting point, while in subsequent iterations, the MLP's predictions are used directly.

```
if iteration == 0: # equiprobable initialisation
    B = np.ones((T,nn))/nn
    #B = np.ones((T,nn))
else:
    B = my_mlp.pred_on_seq(batch_list[n], config['N_classes'] *
                           config['n_states'])
```

A crucial masking operation is then applied to restrict the probability distributions to only those states that are relevant to the current ground truth sequence, followed by normalization to ensure valid probability distributions.

```
mask=[]
for s in range(len(gt)):
    begin = int(gt[s])*config['n_states']
    end = begin + config['n_states']
    mask = mask+[i for i in range(begin,end)]
B = B[:,mask]

sum_prob = np.sum(B,axis=1)
for t in range(T):
    B[t,:] /=sum_prob[t]

try:
    log_B = np.log(B)
except RuntimeError:
    print("log 0 = -inf...")
```

The core HMM computations involve calculating both forward and backward variables using the processed emission probabilities. The forward lattice computes the probability of partial state sequences up to each time step, while the backward lattice computes probabilities from each time step to the end of the sequence.

```
#####  
# Compute forward variables  
fwd_lattice = GT_Model.forward(T,log_B)  
#####  
# Compute backward variables  
bwd_lattice = GT_Model.backward(T,log_B)
```

These computations enable the estimation of the sequence's log-likelihood under the current model parameters through the log-sum-exp operation on the combined forward-backward scores at the final time step. The accumulated log-likelihood across all sequences in the batch serves as a key training metric, indicating how well the current neural network parameters explain the observed data.

```
log_likelihood = logsumexp(fwd_lattice[T-1,:]+bwd_lattice[T-1,:])  
  
LL_train += log_likelihood
```

Finally, the method computes posterior state probabilities using the gamma procedure, which combines information from both forward and backward variables to estimate the probability of being in each state at each time step given the entire observation sequence. These state-level posteriors are then mapped back to symbol-level probabilities through the `GT_statesToSymbol_states` function, producing the final predictions that will guide the neural network parameter updates in the subsequent maximization step.

```
#####  
# Compute the gamma posteriors  
gamma = GT_Model.gamma(T,fwd_lattice,bwd_lattice,log_likelihood)  
  
# then distribute posterior probabilities of the GT states  
# to the symbol states involved in the current GT  
symbol_states_posteriors = GT_statesToSymbol_states(GT_Model,gt,  
                                                    gamma,  
                                                    config['N_classes'])
```

Question 4 : explain the `GT_statesToSymbol_states()` method

Its primary role is to transform the state-level posterior distributions obtained from the composite GT model into a standardized format that can be utilized by the neural network during the subsequent maximization step. That is, it converts the sequence-specific, compact state probabilities (the number of states in the ground truth digit) into a universal, fixed-format target vector (with all the states for all possible digits) that the neural network can learn from.

For every symbol in the GT sequence, the method identifies the symbol range in the output matrix corresponding to all possible states of that symbol type across the entire vocabulary, and the range of the input state posterior matrix corresponding to the specific states allocated to that symbol position within the composite GT model. The mapping operation then transfers the computed posterior probabilities from the composite model's state positions to the appropriate symbol-level positions in the output matrix, effectively redistributing the state-specific probabilities into a unified symbol-state probability space.

For example, the model trained with 5 states creates a correlation between every 5 numbers and one digit for each of the 10 classes. In the prediction shown, we can clearly see the class "0" is composed of the states from 0 to 4, the class 1 is composed of the states from 5 to 10, the class 2 is composed of the states from 10 to 14 and so on.

GT: 72104 recognized 72104
best path [35 35 35 35 35 36 37 37 38 39 39 39 39 10 11 12 13 13 14 14 14 14 14 14
5 5 6 7 8 9 9 9 9 9 0 0 1 1 2 3 3 4 4 20 20 20 21 22]

Question 5: in the `forward()` and `backward()` code what are the `log_B` variables ? What is it for ?

It represents the logarithm of the emission probability matrix, that is, `log_B[t, j]` contains the log-probability that the system, when in state `j` at time `t`, would generate the specific observation (the neural network's output) that we actually see at that time step. These probabilities represent how well the neural network's current predictions explain the observed handwriting digit for each possible HMM state.

In the forward algorithm, `log_B[t, j]` is used to incorporate the evidence from the current observation into the state probability calculation. At each time step `t`, after computing the probability of reaching state `j` from all possible previous states (the transition part), the algorithm multiplies this by the probability that state `j` would emit the current observation. Since we're working in log-space, this becomes an addition:

```
fwdlattice[t, j] = logsumexp(work_buffer) + log_B[t, j]
```

In the backward algorithm, `log_B[t+1, :]` serves a similar purpose but in the reverse direction. When calculating the backward probability from time `t+1` back to time `t`, the algorithm

accounts for the probability that each possible state at time $t+1$ would generate the actual observation at time $t+1$. This is captured in the term:

```
work_buffer = bwdlattice[t+1,:] + log_B[t+1,:] + self.log_A[i,:]
```

Question 6 : Explain the gamma() method

Its primary function is to determine the probability distribution over hidden states at each time step during the E step, given the complete observation sequence and the current model parameters. They represent the model's best guess about the true state alignment based on the current model parameters and the observed data.

The method operates by leveraging the mathematical relationship between the forward and backward variables. For each time step t and each state s , it computes the log-posterior probability by summing the corresponding forward variable $\text{fwd}[t,s]$ and backward variable $\text{bwd}[t,s]$.

```
log_posteriors[t,s] = fwd[t,s] + bwd[t,s]
```

A critical normalization step follows this computation, where each time step's log-posterior values are adjusted by subtracting the log-sum-exp of all states at that time step. This normalization ensures that the probabilities across all states at each time step sum to one, converting the raw log-probabilities into properly scaled posterior distributions.

```
log_posteriors[t,:] -= logsumexp(log_posteriors[t,:])
```

The final operation involves applying the exponential function to transform the normalized log-posteriors back into standard probability values, resulting in a $T \times n_states$ matrix where each element represents the probability of being in a particular state at a specific time given all observations.

```
pred= np.exp(log_posteriors)
```

Question 7 : int the method pred_on_batch() explain the following lines of code ?

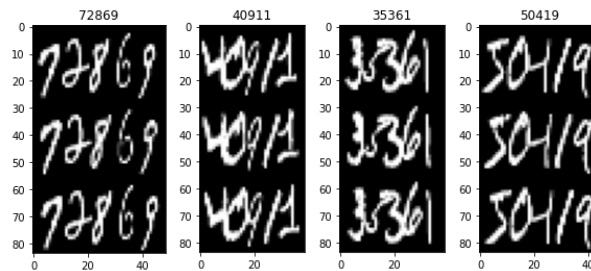
```
sum_prob = np.sum(B,axis=1)
for t in range(T):
    B[t,:] /=sum_prob[t]
```

This normalization ensures that the emission probabilities form valid probability distributions at every time step, which is a fundamental requirement for the forward-backward algorithm to function correctly. Also, this operation compensates for the earlier masking step where certain states were removed from consideration based on the ground truth sequence. The masking operation typically disrupts the original probability distribution, leaving rows that no longer sum to 1.0, and this normalization restores the proper probabilistic structure.

Sliding window

In order to allow the MLP to take account of more context, we introduce a sliding window of size w_width , and stride w_stride . The sliding window takes into account every pixels in the window and is cast into a feature vector of size $28 * w_width$ pixels or features. This sliding window moves left to right over the image with a step of size w_stride . An image of width T is transformed into a sequence of length T/w_stride of feature vectors of size $28 * w_width$.

Some examples with $w_width = 3$ and $w_stride = 2$, providing feature vectors of size 84.



Question 8 : Train a model over 20 epochs with the following configuration:

N_epochs	N_states/symbol	#train seq	batch_size	w_width	W_stride
10	5	2000	128	3	2

Observe the training loss of the MLP and the Log Likelihood of the Neuro_HMM over time. Explain what happen?

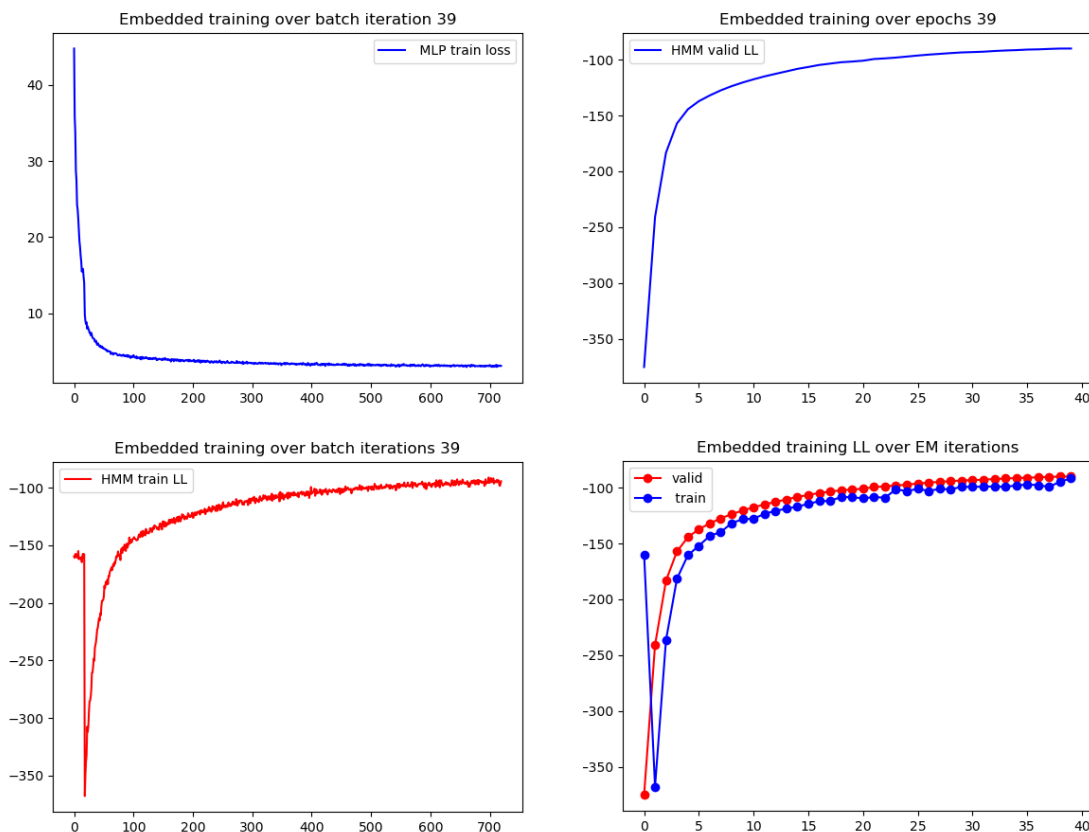


Figura 1 – Curvas de Loss e de log likelihood para treinamento e teste for all epochs durin training

It may be important to highlight the difference in training HMM and MLP, in the first, the objective is to increase LL, while in the second we want to reduce loss . We can see from Figure 1 that the MLP loss and the HMM validation LL curves exhibit the expected behavior. They improve rapidly initially until they approach convergence. However, the still slightly increasing behavior of the LL during validation suggests that training could still improve with an increase in the number of epochs. It is import to

The training LL curve, however, diverges from the usual behavior due to the uniform start, which will be discussed in more detail later. However, we can also point out that the training LL values were almost always lower than the validation values. This is uncommon, but it may simply indicate that the validation datasets were less complex than the training ones.

Question 9: Evaluate the performance of the Neuro_HMM on the test set. Explain the recognition algorithm in detail?

What is the algorithm used, why?

What is the kind of HMM used during decoding, why?

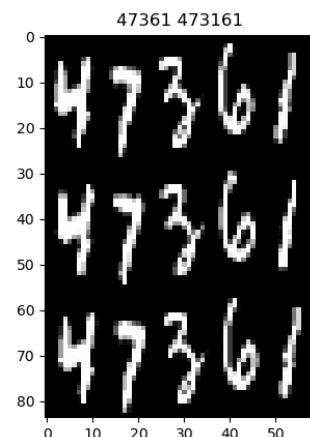
The Viterbi is the algorithm used for decoding, finds the most likely sequence of hidden states (and consequently the most likely sequence of symbols) given the observed input data and the trained model parameters.

For the decoding, the model uses a Ergodic HMM, where from the final state of any digit model, transitions are allowed to the initial state of any other digit model with equal probability, but states within each digit model follow a strict left-to-right progression with self-transitions and transitions to the next state of the same digit. This allows the model to recognize sequences of various lengths, even without knowing the boundaries between the digits.

For example, if we observe one of the predictions paths, we can see that the states transition in a increasing order within the same digits, but only in the frontiers between digits we see a great variation of states. Also, it is evident that each digit has a different number of predicted states, because it varies accordingly to the number of pixels it occupies in the sequence. The number "1" in red didn't existed in the GT, but was predicted in only 5 states.

GT: 47361 recognized 473161

best path [20 20 20 20 20 21 21 22 23 23 24 24 35 35 35 35 36
37 37 38 39 39 39 39 39 15 16 17 17 17 17 18 19 5 6 7 8 9
30 31 32 33 33 33 33 34 34 5 5 5 5 6 7 8 8 9 9 9 9]



Analyzing the results of all predictions for this model, it obtained a character RR of 85,95%, that is lower compared we the models previously discussed in other Tps, but this is a satisfying result, because the task this NeuroHMM performs is harder. This time we are analyzing sequences of slightly overlapping digits, and not digits individually.

FP 1405

TOTAL characters 10000

String FP 1031

TOTAL strings 2000

Recognition ended successfully, Character Error Rate = 14.05 %

, Character Recognition Rate = 85.94999999999999 %

, String Error Rate = 51.55 %

, String Recognition Rate = 48.45 %

Also, we see that the character RR is much Higher than the string RR (48,45 %), that means that model doesn't get as many wrong predictions within a sequence, but most of the sequences contains errors. The performance is evaluated based on the edit distance, every divergence between the GT and the predicted result is considered in the calculation, that is, miss-classifying a digit (substitution), not recognizing a digit (inclusion) or adding a digit to the string (deletion). In the above example we only see one error of recognizing a digit that does not exist.

Question 10: To provide a better initial model with missing data, we chose to process the first E step using a uniform distribution to predict the HMM states in the `pred_on_batch()` method. Observe the role of such initialization by viewing the training loss and compare this initialization with a pure random initialization.

Observing Figure 1, we see that the training initialization exhibits a peculiar behavior. The uniform distribution results in nearly random initial behavior. In the first epoch, the coefficient update causes a significant drop in performance, but the model improves as the epochs progress. Shortly before 100 batches, the model returns to the same level as random, but continues to improve until training convergence is achieved.

Question 11 : Explore training and testing some other models with more and fewer states, and other parameters of the sliding window

	N_states	w_width	W_stride	Character RR	String RR
Model 1	5	3	2	85,95 %	48,45 %
Model 2	3	3	2	54,88 %	11,20 %
Model 3	7	3	2	84,94 %	45,60 %
Model 4	3	5	4	81,99 %	38,70 %

Using the initially trained model, Model 1, the effect of increasing and decreasing N_state was explored. Model 2, with only 3 states, performed considerably worse, with only 54.88% character RR and 11.20% string RR, thus not achieving satisfactory results. This may indicate that the number of states is insufficient to correctly interpret the data.

The increase to 7 states in Model 3 also caused a decrease in performance, but to a lesser extent, maintaining the same level of accuracy as Model 1. It is possible that both models are still statistically equivalent. It is important to note that increasing the number of states also strongly influences the increase in training time and is a factor to be considered when selecting the best model.

Although decreasing the number of states decreases performance, it was observed that increasing the size of the sliding window can compensate for the decrease in states. This may indicate that the more comprehensive window better summarizes the image and thus allows the smaller number of states to obtain all the information necessary to represent the data. A larger window provides a broader spatial context for the neural network at each time step.

Question 12 : Conclusion

- **Discuss the capacity of Embedded Training to train with unknown segmentation.**
- **Discuss the capacity of Neuro_HMM with sliding window to model long range dependencies.**
- **What is the best configuration you would recommend, why?**

The embedded training is very useful for digit sequence recognition without segmentation, because it does not require each digit to be isolated and labeled individually before training, which is a long and costly dataset to create. The Neuro-HMM, learns to segment and recognize the sequences simultaneously.

Based on the results, the best configuration is a balanced one that optimizes performance without unnecessary computational cost. I would recommend the original Model 1 (with 5 states and a small window) over the alternatives. The 5 states provide enough temporal granularity to model the digits effectively without introducing excessive complexity.